

Merton Structural Credit Risk Model: An Empirical Test

User Manual

Ishaan Khambhampati

1. What this project does

This model estimates a publicly listed company's implied PD (probability of default) using solely publicly available market data such as price history, balance sheet debt, and risk-free rate. This is a structural credit model based on Robert Merton's framework in 1974, which served as the theoretical foundation for Moody's KMV.

2. Setup

pip install requests

If setting up sec_debt_fetcher.py, open it and replace the placeholder at the top:

```
USER_AGENT = "REPLACE_ME YourName your_real_email@example.com"
```

with your real name and a real email. This is because the SEC requires this on every request and will return errors (403 Forbidden) without it. Note that this isn't a login credential but rather a verification.

3. File overview

File	Purpose
merton_model.py	This is where the core math happens for the model. There is no need for any additional setup.
example.py	This is an example program based on a single ticker. In this case, the ticker was Vistra Corp. (VST). You can swap this ticker for a different one if you wish to. This will only give the output for one ticker.
screener.py	This runs the model across the selected 25-company shortlist. This is the program you run to get your output.

screener_stress_test.py	This runs the model across a 16-company dataset concentrated in the BB–CCC risk tier. Run this program to get the stress-tested company output.
sec_debt_fetcher.py	This program pulls debt data from the SEC's XBRL filings for a given ticker.
batch_fetch_main25.py	This runs the sec_debt_fetcher.py program across the 25-company list
batch_fetch_stress_test.py	This runs sec_debt_fetcher.py across the 16-company stress list
price_data.py	You can either manually download the Nasdaq CSV files for each ticker price history that you require (required method) or you can run this program to use an automated Alpha Vantage API.

4. Running what already exists

To run the existing 25 company screener, simply enter:

```
python screener.py
```

Into the terminal.

This program requires a TICKERHistoricalData.csv file in the same folder for every company in its COMPANIES list. The naming system is specific, with an example as follows: AAPLHistoricalData.csv. If a file is missing, that company is skipped with a clear message.

It works the same way for the stress test. Just type the following into the terminal:

```
python screener_stress_test.py
```

5. Adding a new company

Step 1: Get the price history (required)

1. Go to nasdaq.com/market-activity/stocks/TICKER/historical (replace TICKER with the real ticker, lowercase)
2. Set the time range to 1 year, daily interval
3. Click Download data
4. Save the file in your project folder as TICKERHistoricalData.csv (e.g. NVDAHistoricalData.csv)
5. I would suggest using Nasdaq almost always instead of the automated API unless you have access to a paid tier. The free-tier APIs like Alpha Vantage, Stooq won't offer you a full year of history for free or will block automated requests with bot-detection tests. That said, the option is still available to you if that is what you wish.

Step 2: Get the debt data (two options)

Option A: automated (try this first)

Open `sec_debt_fetcher.py`, or write a short script:

```
from sec_debt_fetcher import fetch_debt_data

import json

data = fetch_debt_data("TICKER")

print(json.dumps(data, indent=2))
```

Run it. If it works, you'll see `short_term_debt`, `long_term_debt`, and `shares_outstanding`, each with the XBRL tag name and reporting date used. Always check the date is recent (within the last few months). Although the fetcher has a built-in program that rejects values older than ~15 months, still double-check yourself.

This might fail for some companies, and that's expected. A lot of companies, especially ones under actual financial stress or an uncommon capital structure, will not show the results. If you see this error, move to Option B.

Option B: manual (fallback option)

1. Go to nasdaq.com/market-activity/stocks/TICKER/financials
2. Go to the Balance Sheet tab
3. Find the row labeled "Short-Term Debt / Current Portion of Long-Term Debt" and "Long-Term Debt" for the most recent quarter
4. Find shares outstanding for the same company
5. Compute per-share figures:
$$\text{short_term_per_share} = \text{short_term_debt_thousands} \times 1000 / \text{shares_outstanding}$$

$$\text{long_term_per_share} = \text{long_term_debt_thousands} \times 1000 / \text{shares_outstanding}$$

Always double-check: does $(\text{short_term} + \text{long_term}) \times \text{shares_outstanding}$ roughly match the company's known total debt.

Step 3: Add it to a screener

Open the screener program that you are using, and add an entry to the COMPANIES list:

```
{"ticker": "TICKER", "csv_file": "TICKERHistoricalData.csv",  
  "short_term": your value, "long_term": your value},
```

Step 4: Run it

python screener.py

6. The Interpretation

For each company, you will get two probability outputs:

- KMV barrier PD. This uses only short-term debt + 50% of long-term debt as the "danger threshold." The idea behind this is that only near-term debt obligations would be the realistic reason for a trigger to default in the near future.
- Total debt barrier PD. This uses full debt (short + long) as the threshold.

In this project's testing phase, I saw that the total debt barrier tracked real historical default rate benchmarks much better. Therefore, don't immediately assume that the official Moody's KMV standard is automatically the more accurate choice. Test both and understand which methodology fits your sample companies better based on your criteria.

Additionally, a result of exactly 0.0000% is common and expected for genuinely safe companies. This has been documented as a limitation of the Merton model because it can't account for the small-but-real default risk of very safe companies. It instead assumes smooth, continuous price movement with no sudden shocks. Therefore, don't interpret 0.0000% as "this company can never default" but rather as "this model has no signal to offer for this company."

Likewise, a result at or near 100.0000% should also not be read literally. The probability calculation is clamped between 0% and 1%. This is an extreme result that usually means the company's debt-per-share exceeds its equity-per-share by a huge amount. This is a genuine signal of severe distress, but "100%" does not actually mean a literal certainty. Cross-check any extreme result against factual information from news, credit ratings, litigation, and index membership changes) before treating it as meaningful insight.

7. Common errors and fixes

`ModuleNotFoundError: No module named 'requests'` → Run `pip install requests`. If using a virtual environment, make sure it's activated first (`venv\Scripts\activate` on Windows, `source venv/bin/activate` on Mac/Linux).

`requests.exceptions.HTTPError: 403 Client Error` (when running `sec_debt_fetcher.py`) → You haven't set a real `USER_AGENT`. See Section 3.

`FileNotFoundError` or "Skipping — CSV not found" → You haven't downloaded that company's price CSV yet, or it's named incorrectly. Check the exact filename the script expects against what you actually saved.

`ValueError: Need a reasonably long equity series...` → Your price CSV has fewer than 30 rows. Re-download with a full 1-year range, or check the file didn't get truncated.

A company's numbers look wildly implausible (e.g. debt-per-share is 10x higher than the stock price for a company that isn't actually in distress) → Almost always a units error (thousands vs. millions) or a shares-outstanding mismatch (e.g. picking up one share class of a multi-class company, or a stale count from before a recent stock split/spinoff). Redo the sanity check in Step 2 above.

`ImportError` involving `numpy/pandas/DLL` loading (Windows) → This project's core model doesn't need `numpy` or `pandas` at all — if you're hitting this, you likely installed something unrelated that pulled them in. Check what you actually need before installing; `merton_model.py`, `screener.py`, and `sec_debt_fetcher.py` only need `requests` plus Python's standard library.